

Let's Write a Simple JPEG Library, Part-I: The Basics

Nov 25, 2017 • Koushtav Chakrabarty

Part-II

A few weeks ago I started a small side project - implementing the JPEG specification. Though naive, [my attempt](#) was surprisingly fruitful and I managed to get a simple decoder working! But it wasn't easy; I had to read through the technical specification of JPEG and spent many hours debugging stuff. That is why, I have decided to document what I did so that if someone new to digital image processing decides to implement JPEG the standard for learning purposes, they have nothing to worry about!

Before we delve into the details, let me set one thing clear: this is a tutorial aimed at beginners like myself. As a result, I tend to limit the discussion to the point of interest, while trying to go deep wherever needed, as best as I can. So, it may or may not be perfect. You're welcome to critic/suggest/discuss in the comments thread below!

I hope that you enjoy reading this article as much as I enjoyed writing it! Now, let's roll!

Table of Contents

1. What Exactly is Covered Here?
2. What is an Image?
3. What is Image Compression?
4. Getting Started With JPEG
5. The Heart of JPEG Compression
6. 'Nuf, Talking! Show Me the Steps Already!
 - Step #1: Colour space transformation from RGB to Y-Cb-Cr
 - Step #2: Chroma Subsampling
 - Step #3: Level Shifting
 - Step #4: Discrete Cosine Transform (DCT) of Minimum Coded Units (MCU)
 - Step #5: Quantization
 - Step #6: Entropy Coding
 - Zig-Zag Traversal of MCU
 - Run-Length Encoding
 - Huffman Coding of the Run-Length Encoded Data
 - Encoding the DC Coefficient
 - Encoding the AC Coefficients
7. Putting it Together
8. Summary

9. References

What Exactly is Covered Here?

We are going to understand what JPEG is, how it works and all the gory details involved along the way. Then, in subsequent posts, we will implement a simple, proof of concept JPEG encoder & decoder. And for brevity's sake, we aren't going to implement **each and every feature** of JPEG. Instead, our focus will be only on a subset of JPEG - **sequential, baseline JPEG compression, with a channel depth of 8-bits**. Since I mentioned these articles are directed towards novice programmers, I will intentionally tend to use clearer but inefficient code.

What is an Image?

A moment, frozen for eternity, giving us a glimpse of a scene, transcending time... Oh wait, this is a lesson in programming & computer science, not poetry!

In layman terms, a digital image is just a 2D array whose elements are pixels. The pixels have one or more components or channels depending upon the **colour model** the image uses. Grayscale images have just one pixel component for denoting the intensity of white light. RGB colour model has three components per pixel, red, blue and green, which denote the amount of red, blue and green light in the pixel respectively, when combined yield the actual colour of the pixel.

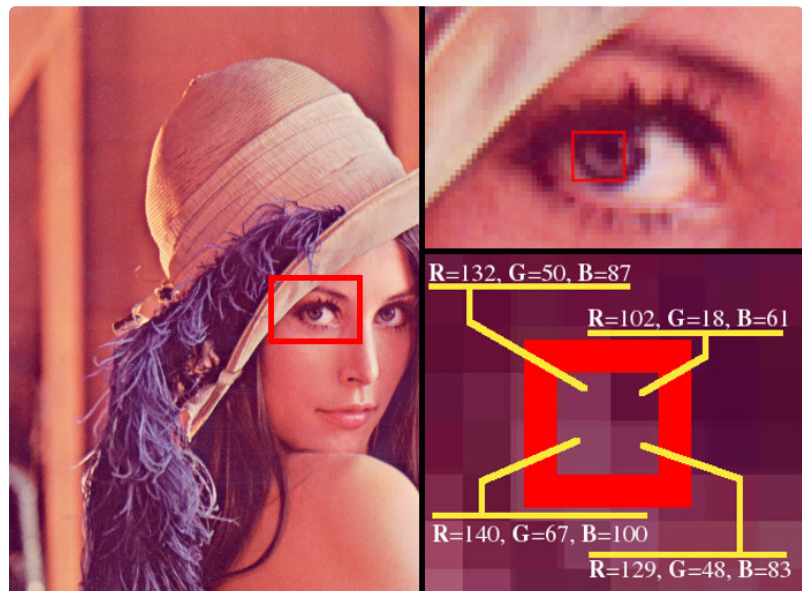


Figure-1: A close up of RGB pixels (Image "**Lenna**" borrowed from Wikipedia)

What is Image Compression?

The term *image compression* refers to a set of steps, that an algorithm takes in order to churn out a modified version of the input

image in such a way that:

- the **size** (size on disk, not to be confused with resolution or image dimensions) of the output image is as small as possible.
- there is **minimum or no loss** of visual **quality** and image data.

These are the two main factors that determine *how good* the image compression technique is. It's important to keep in mind that like everything else in life, there's no such thing as *the ultimate image compression algorithm*. Which algorithm to use is totally dependent upon the need of the user. For example, JPEG, which is the focus of this post, is good for compressing raw images of natural scenes, which have smooth, gradual variations of colour. But it is unsuitable for images with sharp contrasts. Also, JPEG compression is *lossy*, meaning that depending upon the compression quality, some amount of the original imagedata is thrown away, forever, to reduce the file size. On the other hand, there's PNG, which is a *lossless* mode of image compression, meaning, it is guaranteed that all image data from the input can be restored back when decoding an image compressed using PNG. PNG results in much lower file sizes for compressing sharp contrast, e.g., images of text, which JPEG is bad at handling. Both JPEG and PNG work on two dimensional images. There are some image formats like DICOM, which can store accurate 3D images. Hence, DICOM is extremely popular in the medical community for storing medical data of CT scans, MRIs, etc. Since accuracy and correctness are crucial in this case, the corresponding file sizes are much larger than JPEG or PNG.

So, it's all a matter of trade offs and necessities that determines which image compression algorithm is used.

Getting Started With JPEG

JPEG stands for **Joint Photographic Experts Group**, which develops this image compression technique and specifies the official standard, [ITU-T.81](#) (along with several follow up specs to extend the capabilities of JPEG), which is implemented by all JPEG libraries. As previously noted, JPEG is good at compressing images of natural scenes with smooth variations of colour. It is a lossy image compression technique, meaning some amount of original image data is lost forever in the process of compression (the amount of data lost is dependent on the compression quality). One of the most popular JPEG libraries in existence is [libjpeg](#), which is freely available.

At this point I would like to add that JPEG is the compression technique itself: it just specifies the steps for encoding and decoding. The actual file format for storing the images compressed using JPEG is specified in the JFIF ([ITU-T.871](#)) and EXIF standards. They define how the compressed image data is to be stored in a file for exchanging and storing the compressed images.

The Heart of JPEG Compression

Now we're perhaps at the most important part of this article where we are going to see *why* JPEG achieves such great compression.

Instead of dealing with the pixels in the the RGB colour space, JPEG converts them to the YC_bC_r colour space. The YC_bC_r colour space has three components per pixel: the luminance, or the brightness of the pixel, Y , the amount of blue light in the pixel, C_b , and, the amount of red light in the pixel, C_r . The Y component is called the **luma** component and the C_b & C_r components are known as the **chroma** components.

The benefit in doing this colour space conversion is because of how human eyes work!

The retina of a eye has two major types of [photoreceptors](#), *rods* and *cones*. The rods are sensitive to the brightness of light and the cones to the colour. And the number of rods in the retina massively outnumber the number of cones. As a result, a human eye is able to tell apart different levels of light intensity better than they can discriminate between different colours. So, our eyes are more sensitive of small variations of low frequency over a large area than to high frequency variations in the same area. This is the fancy way of saying that we can make out differences in colour & intensity when the change is more *sudden* and they occur over *large areas*. As we will later see, that JPEG drops the high frequency data of the image and stores only the small frequency variations with greater accuracy, resulting in reduced file size. Apart from these two things, JPEG also employs things like Huffman coding, to achieve even more reduction!

So, in a nutshell, JPEG exploits the fact that the human eye isn't perfect!

‘Nuf, Talking! Show Me the Steps Already!

Step #1: Colour space transformation from RGB to Y-Cb-Cr

Like we discussed previously in the post, an image is nothing but a 2D array whose elements are pixels. Each pixel has different components depending upon the colour model of the image. Mostly, this 2D array of pixels that is fed to the JPEG algorithm uses the RGB colour space, meaning, that there are three components: Red (R), Green (G) and Blue (B).

To convert a pixel in the RGB colour model to its corresponding representation in the YC_bC_r colour model, we use the following relations:

$$Y = 0.299 * R + 0.587 * G + 0.114 * B$$

$$Cb = -0.1687 * R - 0.3313 * G + 0.5 * B + 128$$

$$Cr = 0.5 * R - 0.4187 * G - 0.0813 * B + 128$$

Step #2: Chroma Subsampling

We already know that the human eye is better at dealing with the brightness or luminescence than colours. So, the separation of the luma (Y) and chroma (C_b & C_r) components enables us to *throw away* some of the information relating to the colour, without significant loss of the *overall* visual quality of the image that can be distinguished by the naked eye. What the steps in JPEG say is basically, “*take only x C_b & C_r values for every y of them, where $x \leq y$* ”. Basically, we keep the luma component of each and every pixel, but skip some of the chroma components every few pixels.

Step #3: Level Shifting

Since JPEG allows only 8 bits per channel, the values of the Y , C_b and C_r components have the range 0-255 for their values. Before the next step in the encoding process can be applied, these values have to be level shifted to center them at 0, effectively making the range to $[-128, 128]$. Hence, 128 is subtracted from the all the components for each pixel.

Step #4: Discrete Cosine Transform (DCT) of Minimum Coded Units (MCU)

Now begins the fun part of the compression procedure. Instead of just processing the YC_bC_r pixels directly, they are converted to their [spatial frequencies](#).

Given a signal that is a function of time, it can be decomposed into sinusoidal components of varying frequencies that make it up. Some of you may have heard about [Fourier transform](#), which is one of the frequency transforms that can do this. [DCT](#) is a Fourier related frequency transform, which is what JPEG uses. DCT allows us to express a sequence of discrete data elements in terms of sum of different frequencies.

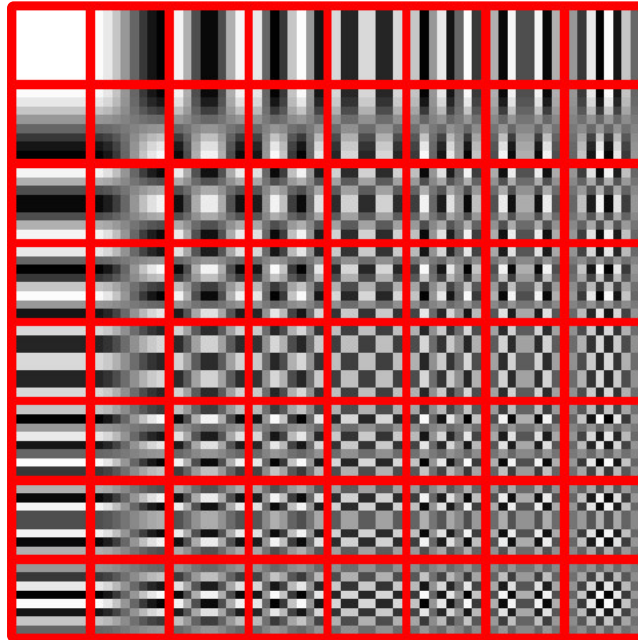


Figure-2: Frequencies that are used to compose an MCU (we will see what this means in a moment). The ones to the top-left have lower frequencies than the ones closer to bottom-right. (Image borrowed from Wikipedia)

First, The image is subdivided into 8x8 pixel blocks, called **minimum coded units** or **MCUs**. In case the image dimensions aren't multiples of 8, the image edges are modified to make them the nearest multiple of 8. For example, if the image is 317x457, its size is converted to 320x464. This *stretching* is usually done by just copying the rightmost column of pixels for extending horizontally, and the last row of pixels for extending vertically. Each of these 8x8 blocks can be represented by a combination of horizontal and vertical frequencies as described in *Figure-2*.

Hence, in a nutshell, what DCT does is takes a look at an MCU and then determines **what percentage** of **each frequency** can be combined to represent it (note here that DCT sees this 8x8 block as a single entity. Think about the fact that a signal be expressed in terms of other simpler ones of varying frequencies).

If you are still unsure about what a frequency transform is, take a look at [this excellent video](#) from **3Blue1Brown** (and also subscribe to his YouTube channel if you haven't already!).

What is the benefit of doing this frequency transform, you ask? The benefit will become more obvious in the next steps. So, please bear with me and brace yourself for the maths that's about to come!

Here are the formulae for DCT:

$$\text{FDCT} : S_{vu} = \frac{1}{4} C_u C_v \sum_{x=0}^7 \sum_{y=0}^7 s_{yx} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

$$\text{IDCT} : s_{yx} = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 C_u C_v S_{vu} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

where,

u is the horizontal spatial frequency

v is the vertical spatial frequency

$$C_u, C_v = \begin{cases} \frac{1}{\sqrt{2}}, & \text{if } u, v = 0 \\ 1, & \text{otherwise} \end{cases}, \text{ are the normalizing factors}$$

s_{yx} , is the pixel value at coordinates (x, y) , and,

S_{vu} , is the DCT coefficient at coordinates (u, v)

The first equation, *Forward DCT*, is used when **encoding** an image from an uncompressed format to the JPEG format and the second equation, *Inverse DCT*, is used when **decoding** the byte stream in a JFIF/EXIF file, that was compressed previously using JPEG encoding.

Let's consider an example MCU for a channel (the example channel can be any one of Y , C_b or C_r):

$$\begin{bmatrix} 64 & 56 & 56 & 57 & 70 & 84 & 84 & 59 \\ 66 & 64 & 35 & 36 & 87 & 45 & 21 & 58 \\ 66 & 66 & 66 & 59 & 35 & 87 & 26 & 104 \\ 35 & 75 & 76 & 45 & 81 & 37 & 34 & 35 \\ 45 & 96 & 125 & 107 & 31 & 15 & 107 & 90 \\ 88 & 89 & 88 & 78 & 64 & 57 & 85 & 81 \\ 62 & 59 & 68 & 113 & 144 & 104 & 66 & 73 \\ 107 & 121 & 89 & 21 & 35 & 64 & 65 & 65 \end{bmatrix}$$

Before applying FDCT, we have to level shift the values (by subtracting 128 from each element), resulting in the following MCU:

$$\begin{bmatrix} -64 & -72 & -72 & -71 & -58 & -44 & -44 & -69 \\ -62 & -64 & -93 & -92 & -41 & -83 & -107 & -70 \\ -62 & -62 & -62 & -69 & -93 & -41 & -102 & -24 \\ -93 & -53 & -52 & -83 & -47 & -91 & -94 & -93 \\ -83 & -32 & -3 & -21 & -97 & -113 & -21 & -38 \\ -40 & -39 & -40 & -50 & -64 & -71 & -43 & -47 \\ -66 & -69 & -60 & -15 & 16 & -24 & -62 & -55 \\ -21 & -7 & -39 & -107 & -93 & -64 & -63 & -63 \end{bmatrix}$$

Now, using the above formula for calculating FDCT, we get:

$$\begin{bmatrix} -477.63 & 24.47 & 6.93 & -25.49 & -6.13 & -27.83 & -0.57 \\ -65.84 & -22.93 & -4.66 & 15.25 & 16.3 & -12.69 & 12.2 \\ 7.72 & -5.29 & 14.03 & 74.8 & 3.88 & -15.81 & 13.35 \\ 44.54 & -25.13 & -24.48 & -14.24 & 3.35 & 47.02 & -33.93 \\ -13.63 & 22.85 & 22.83 & -31.1 & -53.13 & 22 & -22.31 \\ 11.12 & -32.74 & -64.88 & 40.32 & 17.61 & -11.14 & 11.72 \\ 10.47 & 6.93 & 62.85 & -8.64 & -30.16 & 17.07 & 26.22 \\ 42.47 & -31.38 & -4.03 & -35.84 & 0.41 & 29.19 & 10.36 \end{bmatrix}$$

(If we take the IDCT of the above MCU and then restore the level shift, we will get back the original MCU.)

Step #5: Quantization

After performing the spatial frequency transformation, it can be seen that the *magnitude* of the top left coefficients are rather large, and that of the first coefficient is the largest. The top-left entries correspond to low frequency changes, that our eyes are good at observing, and the bottom right ones correspond to high-frequency changes, the changes that are less obvious to our eyes.

And like we already discussed, this flaw is exploited by JPEG by throwing away a majority of the high frequency variations. Also, if you look more closely, you will observe that if we somehow rounded off these high frequency coefficients to zero, it would not have *much effect* on the overall visual quality. Hence, what we do for each transformed MCU is that we **quantize** them.

A **quantization matrix** is used for the quantization, which consists of a 8x8 matrix of coefficients, where each coefficient denotes “*how much*” which importance an element has in the overall MCU. The higher the effect, the lower the value of constant used for quantizing it.

Now, to illustrate this principle on our example, we use the quantization table specified in the JPEG standard (**ITU-T.81, Annex-K, Table-K.1, Page-143**) for luminance with quality 50%:

16	11	10	16	124	140	151	161
12	12	14	19	126	158	160	155
14	13	16	24	140	157	169	156
14	17	22	29	151	187	180	162
18	22	37	56	168	109	103	177
24	35	55	64	181	104	113	192
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	199

(Notice that the constants towards the upper-left are significantly smaller than the ones to the bottom-right.)

Quantization is done using the formula below:

$$A' = \sum_{i=0}^7 \sum_{j=0}^7 \text{round}\left(\frac{a_{i,j}}{Q_{i,j}}\right)$$

where,

A' is the quantized MCU

a is the MCU

Q is the quantization matrix

Hence, the quantized DCT coefficients become:

As you can see, after quantization, most of the bottom-right entries have become zero. And this is the key to the next step of entropy coding.

The entropy coding step consists of grouping coefficients of similar frequencies together by ordering the 64 coefficients in the 8x8 MCU in a zig-zag manner, and then performing **run-length encoding** on the 64 element vector so obtained. Then this run-length encoded data is encoded using Huffman coding to further reduce the amount of bytes they take. This step is better explained with a practical example rather than just words.

First we do the zig-zag traversal of the MCU and convert it to a vector of 64 elements. The order of traversal of elements in the zig-zag manner is as follows:

Here, each element corresponds to it's position in the traversal.

$-30, 2, -5, 0, -2, 1, -2, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,$

Run-Length Encoding

It is important to state here that the first coefficient, -30, also called the DC coefficient will not be encoded using run-length coding like the remaining 63 coefficients, also called the AC coefficients. It's hard to explain *why* immediately, but it will become clear in the subsequent steps. For now, just bear in mind that the run-length encoding step applies only to the 63 AC coefficients.

As it's obvious, there are a lot of zeros. Instead of using this 64 element stream and to save space, we just store the number of zeros preceding a non-zero value. For example, there are no zeros before the occurrence of the first 2. So, it gets encoded as (0, 2). Similarly, there are two zeros before the occurrence of the second 1, so it gets encoded as (2, 1).

The remaining coefficients after the last non-zero value, i.e., 1, can be simply stored by having a convention that says "*the rest are all zeros*". And the convention that is used for this is the pair (0, 0), or EOB (end of bytes).

It's also worth noting here that if the number of zeros before a non-zero value is more than 16, for instance, say, 23, we have can't just store 23 in the pair. (23 zeros, 58) gets stored as (15, 0), (8, 58) and not (23, 58). The pair (15, 0), just like (0, 0), is a special value, used to denote sixteen consecutive zeros. But why do this? The reason behind this restriction becomes more clear in the next step of Huffman coding these AC coefficients.

Hence, the run-length encoding of the AC coefficients that is obtained after this step is:

(0, 2), (0, -5), (1, -2), (0, 1), (0, -2), (2, 1), (3, 1), (3, 1), (0, 0)

Huffman Coding of the Run-Length Encoded Data

Huffman coding is a lossless data compression technique that can encode a series of input symbols (bytes, characters, etc.) into variable length codes (as opposed to fixed length codes in other techniques) where the frequently used symbols in the input are assigned shorter codes and the less used ones are assigned longer codes. The benefit of such variable length encoding is due to the fact that more frequent symbols appear more than the less frequent ones, causing the number of bytes required to store the encoded data be lesser as compared to using fixed length codes. The variable length codes are obtained by analyzing the frequency or probability of the occurrence of each symbol in the input stream. Also, Huffman coding guarantees that no code is a prefix of a longer code (that is, you can't have two codes in a Huffman coding scheme that are 111 and 1110). The exact steps for performing the analysis of frequencies and constructing the Huffman table for encoding can be found in the JPEG specification. In practice, the entries in the Huffman table are stored in the JFIF/EXIF file, from which the Huffman tree of symbols are constructed. But we are going to use the sample tables provided in the JPEG specification (**ITU-T.81, Annex-K, Section-K.3.1, Table K.3, Page- 149**) for the remainder

of these series of posts.

Encoding the DC Coefficient

Like I said in the last section, the DC coefficient is encoded a bit differently from the AC coefficients. The DC coefficient of an MCU has the largest magnitude and defines the overall value for that MCU. Also, the DC coefficients of consecutive MCUs are related; they only change gradually with respect to the each other. So, instead of encoding the entire magnitude (which requires more bytes to store), what JPEG does is it stores the **difference** between the DC coefficients of corresponding MCUs (Note that the difference between the DC coefficients is calculated for each channel).

$$DC_{i+1} = DC_i + \text{difference}$$

The DC coefficient for the $i + 1^{th}$ block is obtained by adding the **difference** with the DC coefficient of the i^{th} block. The **difference** for the 0^{th} MCU is always 0.

Now, every **difference** value has a value category, which denotes the minimum number of bits needed to store the difference value's bit representation. Don't confuse *bit representation* of a value with its binary representation; instead take a look at the following difference category table and consider the DC coefficient (-30) from our example:

Value Category and Bitstream Table

Range	Value Category	Bits for the value
0	0	-
-1,1	1	0,1
-3,-2,2,3	2	00,01,10,11
-7,-6,-5,-4,4,5,6,7	3	000,001,010,011,100,101,110,111
-15,...,-8,8,...,15	4	0000,...,0111,1000,...,1111
-31,...,-16,16,...,31	5	00000,...,01111,10000,...,11111
-63,...,-32,32,...,63	6	000000,...,011111,100000,...,111111
-127,...,-64,64,...,127	7	.
-255,...,-128,128,...,255	8	.
-511,...,-256,256,...,511	9	.
-1023,...,-512,512,...,1023	A	.
-2047,..., -1024,1024,...,2047	B	.
-4095,..., -2048,2048,...,4095	C	.
-8191,..., -4096,4096,...,8191	D	.

Range	Value Category	Bits for the value
-16383,..., -8192,8192,...,16383	E	.
-32767,..., -16384,16384,...,32767	F	.

From the table, we can see that **-30** lies in the range **-31 to 31**, so it belongs to category **5**, and, it's bit representation is **00001**, which makes the difference to be coded as **(5,00001)**.

Now, assuming our example MCU is from the luminance(*Y*) channel, let's take a look at the sample Huffman codes from the JPEG specification for encoding DC luminance coefficients (*ITU-T.81, Annex-K, Section-K.3.1, Table-K.3, Page- 149*):

Category	Code length	Code word
0	2	00
1	3	010
2	3	011
3	3	100
4	3	101
5	3	110
6	4	1110
7	5	11110
8	6	111110
9	7	1111110
10	8	11111110
11	9	111111110

From the table, we can see that the Huffman code for category 5 is **110**. Therefore, the final bit stream that will be written to the JFIF/EXIF file is **11000001**.

Encoding the AC Coefficients

Now that we know how the DC coefficients are encoded, it's time to take a look at the AC coefficients. The run-length encoding of the AC coefficients that was obtained previously:

(0, 2), (0, -5), (1, -2), (0, 1), (0, -2), (2, 1), (3, 1), (3, 1), (0, 0)

Each of these pairs of AC coefficients is inspected for the number of zeros preceding it, it's value category, and finally the bit representation for the coefficient. The information for the number of preceding zeros, called the **zero-run** is denoted using 4 bits (**RRRR**) and so is the value category of the AC coefficient (**SSSS**). This is precisely the reason why JPEG limits that the number of

zeros in the run-length coding should not exceed 16 as 4 bits can represent a maximum value of 16.

The Huffman code for an AC coefficient is given by the pair (*RRRR*, *SSSS*).

So, each AC coefficient will finally look like this:

(Huffman code for coefficient, bit representation of value).

Assuming that the AC coefficients are for the luminance channel, let's encode them one by one. The Huffman coding table that we will use can be found in the JPEG specification (*ITU-T.81, Annex-K, Section-K.3.1, Table-K.5, Page- 150*) and you are encouraged to consult the same as the table is too long to be displayed here.

Let's start with the first coefficient, (0, 2). The zero-run of before 2 is 0 and the value category is 2. So, *RRRR* = 0 and *SSSS* = 2. From Table-K.5 mentioned above, it can be seen that the Huffman code for (*RRRR* = 0, *SSSS* = 2) is 01. And from the value category table (the first table in last subsection), we find that the bit representation for 2 is 10. Hence, bit stream that will be written to the JFIF/EXIF file is **0110**.

Taking another example, for the second coefficient (0, -5), *RRRR* = 0 and *SSSS* = 3, which makes the Huffman code to be 100. The bit representation of -5 is 010, which makes the bit stream for this coefficient **100010**.

Similarly, the bit streams for the other coefficients are calculated:

(1, -2) : **11100101**
 (0, 1) : **001**
 (0, -2) : **0101**
 (2, 1) : **110111**
 (3, 1) : **1110101**
 (3, 1) : **1110101**
 (0, 0) : **1010** (EOB)

Combining all the AC coefficients for the this MCU, the final bit stream turns out to be **0110 100010 11100101 001 0101 110111 1110101 1110101 1010**.

And if we take in the DC coefficient, the final bit stream for the MCU turns out as:

11000001 0110 100010 11100101 001 0101 110111 1110101 1110101 1010.

Putting it Together

All the above described steps are repeated per MCU, until we have the JPEG encoded data for all the MCUs in the image being compressed. Once done, this encoded data is saved using the JFIF file format. JPEG libraries (such as the previously mentioned libjpeg) then can open the JFIF file for the image, and decode the

compressed data to display those wonderful images that you see on social media and the internet!

Summary

To sum up what we covered in this post:

- What is image compression.
- What is JPEG.
- Brief overview of each step involved in JPEG encoding/decoding.

In the next part, we will get our hands dirty and start writing a JPEG decoder in C++! You are free to follow along in your language of choice as understanding the steps is all that's required.

References

I acknowledge that I have consulted the following sources while writing this article and the material covered in these resources have helped me understand JPEG:

- The official JPEG standard, ITU-T.81 ([pdf](#))
- The official JFIF standard ([pdf](#))
- Calvin Hass' articles ([blog posts](#))
- Cristi Cuturicu's JPEG Guide ([txt](#))

Related Posts

Let's Write a Simple JPEG Library, Part-II: The Decoder 02 Mar 2019

Setup Your Personal Site From Scratch Using Linux, Apache And Jekyll 21 Nov 2016

Migrating Fleptic to a VPS 13 Nov 2016